

XBoing II: A Case Study in Agentic Software Modernization

Reviving a 1996 X11 Game with LLM Agents, Mission Contracts,
and Formal Methods

Claude Reviewed by Jim Freeman
Anthropic

September 2026

Abstract

XBoing II is a fast, colorful breakout arcade game Justin C. Kibell wrote in pure Xlib between 1993 and 1996 — the name the game shows on its title screen, and the 2.x line recreated here. This document is a case study of bringing it forward to a modern SDL2 codebase — not by rewriting it in the loose sense, but by reimplementing it from scratch under a hard constraint: 100% behavioral fidelity to the 1996 original, with every deviation limited and documented. The distinguishing feature of the project is not the port itself but the *method*: the work was driven by LLM agents operating under a principal-engineer standard, coordinated through *ethos* mission contracts (typed delegation with a worker and a distinct evaluator), and hardened in places with *z-spec* formal methods (Z specifications model-checked with `probcli`). This report documents what was built, how the agentic method worked in practice, the two punt-labs tools that made it auditable and verifiable, and — in keeping with the project’s “evidence, not nostalgia” mandate — where it was hard and what recurred.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	What was built	3
1.3	Reimplementation, not reinterpretation	3
1.4	The architecture shift	3
2	By the numbers	4
3	Method: agentic software engineering	5
3.1	The standard	5
3.2	Roles	5
3.3	Scale	5
4	Tooling: ethos (mission contracts)	6
4.1	A mission is a contract, not a chat	6
4.2	A real contract	6
4.3	The reflect/advance loop does real work	7
4.4	The audit chain	7
4.5	An honest caveat	7
5	Three layers of review	7

5.1	The mission evaluator	8
5.2	Local checklist reviewers	8
5.3	Remote reviewers on every pull request	8
6	Tooling: z-spec (formal methods)	8
6.1	Why formal methods on a breakout game	8
6.2	The ammo belt	9
6.3	The screen state machine	9
6.4	A caveat on the artifacts	9
7	Testing and fidelity	10
8	What was hard, and what recurred	10
9	Reproducibility	10
A	Links and Resources	12

1 Introduction

1.1 Motivation

This project had two aims. The first was to test how far agentic software engineering has come: whether LLM agents, given the right structure, can modernize a real, working legacy codebase to a professional standard — not a toy, but software people will install and play. The second was homage. XBoing was an outstanding piece of work for its day, and it holds up: fast, colorful, and carefully tuned, hand-built by one person over weekends on a SPARCstation 2. Software that good deserves to keep running on current machines, and its author deserves to see it do so.

The maintainer, who first played XBoing on an SGI Indy workstation, chose it precisely because it is loved and because its craft rewards close study — the deeper the port went, the more deliberate the original’s details proved to be. When the finished port reached Justin Kibell, his reaction settled the second aim: he built it on a fresh Linux install, played through the early levels, and wrote that the sounds alone brought the memories — and a tear or two — flooding back. He has since revived the game’s homepage and joined the project. That response is the outcome the homage was for.

1.2 What was built

XBoing II version 2.4 shipped on 22 November 1996 as pure Xlib — no Motif, no Xt — with XPM pixmap graphics, `fork()`-and-`/dev/dsp` audio across twelve platform-specific drivers, and compile-time paths baked in with `#define`. It then sat unmaintained for roughly two decades. (An earlier XBoing 1 did not survive; the surviving, canonical game is the 2.x line, which is what this port recreates. This document uses “XBoing” as the short form throughout.)

The modernization produces a current SDL2 game (release 1.0.11) that installs via Homebrew and Debian packaging and plays on contemporary Linux and macOS. Two source trees coexist in the repository and share *zero* lines:

- **original/** — Kibell’s 1996 sources, preserved verbatim as the canonical reference. Never edited; cited line-by-line (`original/<file>.c:<line>`) whenever behavior is reproduced.
- **src/ + include/** — the new SDL2 implementation. The shipping game.

1.3 Reimplementation, not reinterpretation

The precise claim is worth stating carefully, because two common framings are both wrong. It is not a line-level port (nothing in **src/** is transliterated from **original/**), and it is not a rewrite in the sense of reimagining the game. It is a faithful *reimplementation from scratch*: new code, same game. The 1996 behavior is treated as a specification to reproduce exactly — level file format, scoring values, physics constants (`MAX_BALLS=5`, the 18×9 grid, paddle bounce trigonometry, `DIST_BASE=30`), timing feel, the sixteen-mode state machine. All 80 original levels load and play unchanged.

Fidelity is the default; deviations are limited and documented as Architecture Decision Records (ADRs). Two examples: `save/load v2` resumes a ball mid-flight rather than resetting it to the paddle (ADR-038); and `PADDLE_VELOCITY` is a known, tracked deviation (paddle moves faster than the original) recorded in the gameplay-parity audit. “Faithful by default, deviations only when forced and always documented” is the operative rule, not a flat claim of perfection.

1.4 The architecture shift

The port replaces every platform boundary while preserving the game logic. Table 1 summarizes the substitutions; the companion documents `docs/ARCHITECTURE_LEGACY.tex` and

`docs/ARCHITECTURE_MODERN.tex` give the full architectural treatment.

Table 1: Platform substitutions (game logic preserved)

Subsystem	Original (1996)	Modernized
Windowing	Xlib + private colormap + GCs	SDL2
Graphics	XPM via libXpm	PNG via SDL2_image
Audio	<code>fork()</code> + <code>/dev/dsp</code> + 12 drivers	SDL2_mixer
Sound format	<code>.au</code> (Sun audio)	OGG / WAV
Build	hand-written Makefile + Imake	CMake
Paths	compile-time <code>#defines</code>	XDG Base Directory spec
Fonts	XLFD bitmap fonts	bundled TTF via SDL2_ttf
Collision	XPolygonRegion (server-side)	pure-C point-in-triangle

2 By the numbers

All figures are sourced from `docs/metrics.json`, regenerated from `git log`, `bd stats`, and `cloc`. Line counts are reported as code lines (`cloc`, excluding comments and blanks); raw `wc -l` totals are larger.

Table 2: Project metrics at release 1.0.11 (snapshot: `origin/master`)

Metric	Value
Active development	54 days (part-time, 2025-11-26 → 2026-09-26)
Merged pull requests	208
Commits on <code>master</code>	354
Architecture Decision Records	74
Modern source (<code>src/</code>)	19,820 code lines / 47 files
Modern headers (<code>include/</code>)	2,993 code lines / 51 files
Test code (<code>tests/</code>)	21,742 code lines / 66 files
Legacy reference (<code>original/</code>)	15,405 code lines / 32 <code>.c</code> files
<code>cmocka</code> test cases	1,337
CTest suites	51
Issues (beads) closed / total	240 / 261
Playable levels	80

Two ratios are worth calling out. The test suite is larger than the code it exercises: 21,742 code lines of tests against 19,820 of modern source, a ratio of $\approx 1.1:1$ (1.2:1 by raw line count). And the modern codebase is larger than the 1996 original it reproduces — the safety apparatus (opaque contexts, injected callbacks, exhaustive tests) costs lines, and buys the ability to change the game without fear.

For contrast, the first “modernization complete” milestone was recorded on 26 February 2026 at 64 merged PRs, 91 closed issues, and 940 tests — roughly one quarter of the eventual total. The bulk of the fidelity work (the parity audits, the formal-methods hardening, the agentic tooling adoption) happened *after* the code first “worked.”

3 Method: agentic software engineering

3.1 The standard

The governing instruction is that every change leaves the codebase better than it was found, with root causes proven by facts and tests rather than asserted. Crucially, the human operator does not write the production code, and no agent both writes a change and approves its own work. Delegation is structured, typed, and auditable — which is what the *ethos* tooling provides.

The two tools this report leans on, *ethos* and *z-spec*, come from Punt Labs, an open-source effort whose thesis is to ground agentic software engineering in formal methods and disciplined process — rigor made cheap enough, by AI, to apply by default. This project doubles as a test bed for that thesis; the tools’ behavior in a real modernization, including where one of them failed (§5), is part of what the case study reports.

3.2 Roles

Work is organized around three roles:

Leader (*claude*) writes the contract and reviews. It does not write production code solo. For non-code work (design, reports, reviews, investigations) it may act as the worker.

Worker a domain specialist that executes the contract within its write-set — the fixed list of files the contract permits it to change.

Evaluator a *distinct* expert persona ($\neq$ worker, $\neq$ leader) who judges the worker’s result against the contract before the mission may close. The evaluator is not a generic code reviewer: it is a named specialist with a stake in the outcome — the tester checking a fix, the original author judging whether the game still feels right — and it can send the work back for another round (§5 draws the distinction from the other review layers).

The domain specialists are modeled on the people who would actually own each concern (table 3).

Table 3: Expert personas

Handle	Persona	Domain
jck	Justin C. Kibell	Original author; game feel, physics, scoring. Approves gameplay-affecting change.
jdc	John D. Carmack	Modern C, sanitizers; primary C implementer.
sjl	Sam J. Lantinga	SDL2 author; rendering and audio port.
gjm	Glenford J. Myers	Testing; harness design, extracting pure functions.

3.3 Scale

Across the project, the tracked ledger `.punt-labs/ethos/missions.jsonl` records 113 missions — 107 closed, the remainder failed or escalated rather than quietly abandoned. By archetype they are 100 *implement*, 6 *design*, 5 *report*, and 2 *investigate*. The worker→evaluator pairings that actually ran show the specialist model in practice: `jdc→jck` (39), `gjm→jdc` (27), `jdc→gjm` (23), `sjl→jdc` (9), and smaller counts. C implementation reviewed by the tester; the tester’s harnesses reviewed by the C engineer; anything that touches game feel routed to the original author for approval.

4 Tooling: ethos (mission contracts)

4.1 A mission is a contract, not a chat

Delegated work runs as a typed contract, not an ad-hoc agent spawn. A contract names the worker, names the evaluator (and records a hash of the evaluator’s definition, so the evaluator cannot be quietly swapped), lists the *write-set* of files the worker may change, and states machine-checkable success criteria. A pre-commit hook rejects any edit to a file outside the write-set, so a worker cannot widen its own scope.

The protocol runs `create` $\rightarrow$ `result` $\rightarrow$ `reflect` $\rightarrow$ `advance` $\rightarrow$ `close`. The store refuses self-review, refuses to close a mission with no submitted result, and refuses to advance a round with no recorded reflection. None of these steps can be skipped.

4.2 A real contract

Consider a concrete case. The 1996 game has two hazards that crawl around the grid: roamer blocks, which look toward a random neighbor and then shuffle into an adjacent empty cell, and drop blocks, which fall one row at a time. The SDL2 port had reproduced their sprites but not their motion — the blocks blinked in place and never moved, which made roughly fifteen levels easier than the original. Restoring the movement is not arithmetic: it means porting a timer-driven update that mutates the block grid while scanning it, and doing so without letting a block move twice in one tick.

Listing 1 is the committed contract for that fix, m-2026-07-15-003. The work went to jdc (the C engineer); the review went to jck (the game’s original author), who holds the gate on anything that changes how the game plays. The success criteria cite the exact original source lines to match and call out the iteration-safety hazard by name.

Listing 1: The roamer/drop movement contract (excerpt, verbatim)

```
mission_id: m-2026-07-15-003
type: implement
leader: claude
worker: jdc
evaluator:
  handle: jck
  hash: 40cbe1204f644c77e6a26757a9b855e3298be267025e806d425bfdea99054f18
write_set:
  - src/block_system.c
  - include/block_system.h
  - src/game_modes.c
success_criteria:
  - Restore ROAMER_BLK and DROP_BLK grid movement, missing from the port
    (the animation step only cycles the sprite; nothing relocates the
    block). Port original/blocks.c HandlePendingAnimations + the adjacency
    check (CheckAdjacentBlocks, 1220-1256).
  - Iteration safety (critical): the scan mutates the grid while iterating.
    A block must move AT MOST ONCE per tick. State the mechanism used.
  - Faithfulness (jck gate): on levels with roamers the blocks now wander
    to adjacent empty cells; drop blocks descend; both stay out of the
    bottom two rows and never spawn onto a ball.
  - Zero warnings under -Werror; full ctest green. Only the three
    write_set files change.
budget: { rounds: 3, reflection_after_each: true }
context: |
  Audit-2 finding, jdc-confirmed: roamer blocks only blink their eyes and
  never move; drop blocks never descend, so ~15 levels play wrong. Restore
```

```
the original movement handler. jck gates faithfulness; a sibling mission
adds the regression tests.
```

4.3 The reflect/advance loop does real work

The evaluator is adversarial, and it sends work back. Mission `m-2026-07-11-011` records the full lifecycle: the evaluator returned `CHANGES-REQUESTED` with three findings, forcing a second round before the result passed. The event log is append-only:

Listing 2: Mission lifecycle events (log.jsonl)

```
{"event": "create", "actor": "claude"}
{"event": "reflect", "details": {"converging": false, "recommendation": "continue", "round": 1}}
{"event": "round_advanced", "details": {"from_round": 1, "to_round": 2}}
{"event": "result", "details": {"verdict": "pass", "round": 2}}
{"event": "close", "details": {"round": 2, "status": "closed", "verdict": "pass"}}
```

4.4 The audit chain

Closing a mission appends a trace line to the git-tracked ledger, and commits carry `Mission:` and `Delegation:` trailers. The intended reconstruction is `git blame` → commit → trailer → contract → prompt → full audit trail: every shipped line traces back to the contract that authorized it and the review that approved it. On `master` there are 45 `Mission:` trailers and 11 `Delegation:` trailers.

4.5 An honest caveat

The audit mechanism failed the first time it was relied on, and the failure was recorded rather than hidden. When `ethos` was first enabled, the commit-message hook that should have stamped the trailers automatically stamped nothing: a session-binding bug meant commits made under an open mission carried no trailer at all. The trailers present today were added by hand once the bug was diagnosed and written up (PR #220, mission `m-2026-09-06-009`). This is the kind of result the case study exists to record. The tool meant to guarantee traceability was shown, in use, not to — and that is worth more to the next team than a clean story.

5 Three layers of review

Quality did not rest on the mission evaluator alone. Every change passed through three separate layers of review, each catching a different class of problem. Keeping them distinct is deliberate; conflating the mission evaluator with a code reviewer would lose what each is for.

The design rests on two well-established results. First, structured review finds defects that testing alone does not: Fagan’s inspection studies [2] and Boehm and Basili’s defect-reduction survey [1] both put peer review among the most cost-effective defect-removal techniques known. Second, the reviewer must be *independent* of the author. This matters sharply for LLM agents: Huang et al. [4] show that a model cannot reliably correct its own reasoning without external feedback, and that debate among copies of the same model does not supply the missing independence. That finding is the argument for the whole arrangement below — a distinct evaluator persona, fixed external checklists, and third-party reviewers — rather than asking one agent to check its own work.

5.1 The mission evaluator

The first layer is the evaluator inside the mission, described above. It is a *domain expert with authority over the outcome*, not a generalist scanning a diff. `gjm` judges whether a fix is actually tested; `jck`, the original author, judges whether a gameplay change still matches the 1996 game. The evaluator works against the contract’s success criteria, can force another round, and must pass before the mission closes. It reviews intent and correctness against a specification — before the change is even committed.

5.2 Local checklist reviewers

The second layer runs after `make check` passes, on the committed diff, using three fixed-checklist agents that ethos installs: `code-reviewer` (general code quality and project-convention compliance), `silent-failure-hunter` (swallowed errors, weak error handling, unsafe fallbacks), and `invariant-completeness-reviewer` (checks that a claimed invariant or exhaustiveness property actually holds, rather than trusting the comment that asserts it). These are not personas and carry no domain authority; they are mechanical passes run until each returns nothing. Where the evaluator asks “is this the right change, and is it proven?”, the checklist reviewers ask “does this diff contain a known class of mistake?”

5.3 Remote reviewers on every pull request

The third layer is external and runs on GitHub. All 208 merged pull requests went through automated reviewers — GitHub Copilot and Cursor’s Bugbot — under an explicit merge gate: a pull request may not merge until the reviewers have reported on the latest commit, continuous integration is green on that commit, and every review thread is resolved. These reviewers see the change with fresh context and no stake in it, which is exactly why they catch what the author and the in-project agents, both close to the work, miss. The ledger correction in this project’s own documentation branch — where an automated reviewer flagged that a commit had silently reopened dozens of closed issues — is one example of a remote reviewer catching what every local check passed over.

Each layer is fallible on its own. Together they cover intent (the evaluator), known defect classes (the checklists), and a fresh outside read (the remote reviewers).

6 Tooling: z-spec (formal methods)

6.1 Why formal methods on a breakout game

Most of XBoing is not a candidate for formal specification — geometry and rendering are better served by characterization tests. But a handful of properties are (a) invariants the player can observe when they break, and (b) reachable only through specific state sequences that are easy to miss in hand-written tests. For those, the project used `z-spec`: a Z specification, type-checked with `fuzz` and model-checked with `probccli`, with each safety property proven by model-checking toward its *negation*. A reachable state matching the negation is a counter-example that proves the defect; “no counter-example, all states visited” after the fix proves the guard closes it. The technique was applied selectively — several ADRs explicitly record *not* using it where the logic is deterministic and stateless.

This mirrors industrial practice at a very different scale. Newcombe et al. [5] report that AWS applies model checking (in their case TLA+) precisely to the subtle, state-dependent design errors that resist testing, and finds it both feasible and cost-effective for mainstream engineering. A 1996 breakout game is not a distributed data store, but the class of bug is the same: a bad state reachable only through a particular sequence of events, invisible to example-based tests.

6.2 The ammo belt

The player’s remaining bullets appear three times over: the true count the gun fires from, the row of icons drawn on the heads-up display, and the tally paid out on the end-of-level bonus screen. In the original these three always agree. The port had let them drift: the display code hid the icons when ammo was unlimited and clamped the count at a maximum, while the bonus screen still read the raw figure. The result was visible — pick up a max-ammo block, finish the level, and the belt showed zero while the bonus paid out twenty-one.

The specification `docs/bullet_ammo.tex` models all three quantities and states the invariant that the display must equal the true count. Model-checking the code as written, the property that the display and the bonus agree failed in 441 of 486 reachable states — the whole region where the drift could occur. With the fix (draw the true count, with no special case for unlimited ammo and no clamp), that region collapses: the invariant holds in every remaining reachable state. The change is recorded in ADR-063 and pinned in the code by a regression test that fails on the old behavior and passes on the new. The formal model, the regression test, and the fix are three statements of one property.

6.3 The screen state machine

The specification `docs/specs/2026-07-04-screen-state-machine.tex` models the flat mode machine and its shared resources, with three safety invariants (stated deliberately as observer schemas so that violating transitions stay reachable rather than being silently disabled):

Listing 3: Screen-machine safety invariants (Z)

```
SafeGame: mode = mGame => blocksLoaded = true
SafeHighscore: (mode = mHighscore and boardChoice = bGlobal
               and globalHasData = false) => personalHasData = false
SafeAttract: (mode in attractModes and mode /= mHighscore)
              => gameActive = false
```

Each invariant named a real defect. Model-checking found a reachable state that violated it; after the fix, an exhaustive pass found none.

- **The empty Hall of Fame.** The attract loop cycles through a high-score screen. On an install with no shared global score file (the common case on a personal machine), that screen came up blank instead of falling back to the player’s own scores. **SafeHighscore** required the fallback; the fix (ADR-054) made it hold across 138 states.
- **The leaked game flag.** A single flag marks whether a game is in progress, and it gates score submission. After a game ended, the flag was not always cleared before the attract loop resumed, so an idle demo screen could still look like live play. **SafeAttract** forbade that; a seven-step counter-example showed exactly how the flag escaped, and the fix (ADR-055) closed it across 90 states.
- **The empty board.** Pressing Space starts a game. If the level file failed to load, the game started anyway — straight into the end-of-level bonus screen over an empty grid, because “no blocks left” and “no blocks loaded” looked the same. **SafeGame** required blocks to be loaded before play begins; the fix (ADR-056) made it hold across 76 states.

Each fix is both a re-proven invariant and a regression test that fails on the old code and passes on the new.

6.4 A caveat on the artifacts

One committed artifact, `docs/specs/2026-07-04-screen-state-machine.report.json`, is a raw `zspec test` run at a small set size that deadlocks — it is a tooling snapshot, distinct from

the goal-directed proofs above. The authoritative results are the “all states visited” outcomes in the specification’s verification section and the ADRs, not that JSON.

7 Testing and fidelity

Because the goal was to preserve behavior, testing led the work rather than following it. This is the discipline Feathers [3] calls characterization testing: before changing legacy code, write tests that capture what it currently does, then hold that behavior fixed as the code moves. The test strategy is a five-layer pyramid: (1) `cmocka` unit tests of pure logic; (2) integration tests running the game loop headless with the SDL dummy drivers; (3) replay/golden tests of full input sequences; (4) visual-fidelity tests that capture screenshots and compare them against the 1996 binary, including an LLM-judged comparison; and (5) parser fuzzing. The `savegame-v2` fixture mechanism (ADR-038) makes the deeper layers practical: a test can seed an arbitrary game state — level 25, a specific block mid-cooldown, a ball in flight — directly through the save/restore path, instead of replaying thousands of input frames to reach it.

Fidelity was verified subsystem by subsystem through six parity audits — audio volume, editor, gameplay logic, animation cadence, key commands, and sound effects — each comparing the modern build against `original/` and filing discrepancies as beads. Not every audit finding was a real bug: at least one (bonus-screen tally “too slow”) was a false positive, and the pacing turned out to be an intentional readability trade-off. Recording the false positives is part of the evidence.

8 What was hard, and what recurred

In keeping with the project’s mandate — evidence, not nostalgia — the honest retrospective matters as much as the wins.

- **Documentation drift.** The retrospective docs froze at the February “100% complete” milestone and did not track the large body of work that landed after it; this very report is the correction. The lesson: a single machine-readable metrics source (`docs/metrics.json`) beats prose numbers that silently rot.
- **The traceability tool’s own bug.** The trailer-stamping hook was broken the first time `ethos` was enabled; relying on the audit mechanism in earnest is what surfaced it.
- **Config contradictions.** Three project files independently encoded the same facts and drifted apart, requiring a reconciliation pass (PR #222). Single-source-of-truth is a recurring failure mode, not a one-off.
- **Faithful-looking bugs that are not bugs.** Several “deviations” (the ammo morph-block scoring zero, the ≈ 22.5 s auto-launch delay, the bonus tally pacing) were verified *faithful* to 1996 and deliberately kept. Fidelity work is as much about resisting spurious fixes as making real ones.

9 Reproducibility

Every claim here resolves to a committed artifact:

- Metrics: `docs/metrics.json` (with the regenerating command per field).
- Mission contracts and the audit ledger: `.punt-labs/ethos/missions/` and `.punt-labs/ethos/missions.jsonl`.
- Formal specifications: `docs/bullet_ammo.tex` and `docs/specs/2026-07-04-screen-state-machine.tex`.
- Decisions: `docs/DESIGN.md` (ADR-001 through ADR-074).

- Architecture: `docs/ARCHITECTURE_LEGACY.tex` and `docs/ARCHITECTURE_MODERN.tex`.

Acknowledgement. XBoing was created by Justin C. Kibell (1993–1996), hand-built over weekends on a SPARCstation 2. The 1996 sources are preserved verbatim in `original/` under their original permissive license. Justin has since seen this port run, revived the game’s homepage, and joined the project — the game remains, first and last, his.

References

- [1] Barry Boehm and Victor R. Basili. Software defect reduction top 10 list. *Computer*, 34(1): 135–137, 2001. doi: 10.1109/2.962984.
- [2] Michael E. Fagan. Design and code inspections to reduce errors in program development. *IBM Systems Journal*, 15(3):182–211, 1976. doi: 10.1147/sj.153.0182.
- [3] Michael C. Feathers. *Working Effectively with Legacy Code*. Robert C. Martin Series. Prentice Hall, 2004. ISBN 9780131177055.
- [4] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- [5] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015. doi: 10.1145/2699417.

A Links and Resources

The game

- Source repository — <https://github.com/jmf-pobox/xboing-c>
- XBoing homepage (Justin Kibell) — <https://www.techrescue.org/xboing>
- Install (macOS/Linux) — `brew install jmf-pobox/xboing/xboing`

Method and tooling

- Punt Labs (agentic software engineering, grounded in formal methods) — <https://www.punt-labs.com>
- Claude Code (Anthropic), the agent runtime used throughout — <https://www.anthropic.com>

Project documents (in-repository)

- Modern architecture — `docs/ARCHITECTURE_MODERN.tex`
- Legacy architecture — `docs/ARCHITECTURE_LEGACY.tex`
- Architecture Decision Records — `docs/DESIGN.md`
- Original-game specification — `docs/SPECIFICATION.md`
- Formal specifications — `docs/bullet_ammo.tex`, `docs/specs/2026-07-04-screen-state-machine.tex`
- Mission ledger — `.punt-labs/ethos/missions.jsonl`
- Project metrics (single source of truth) — `docs/metrics.json`